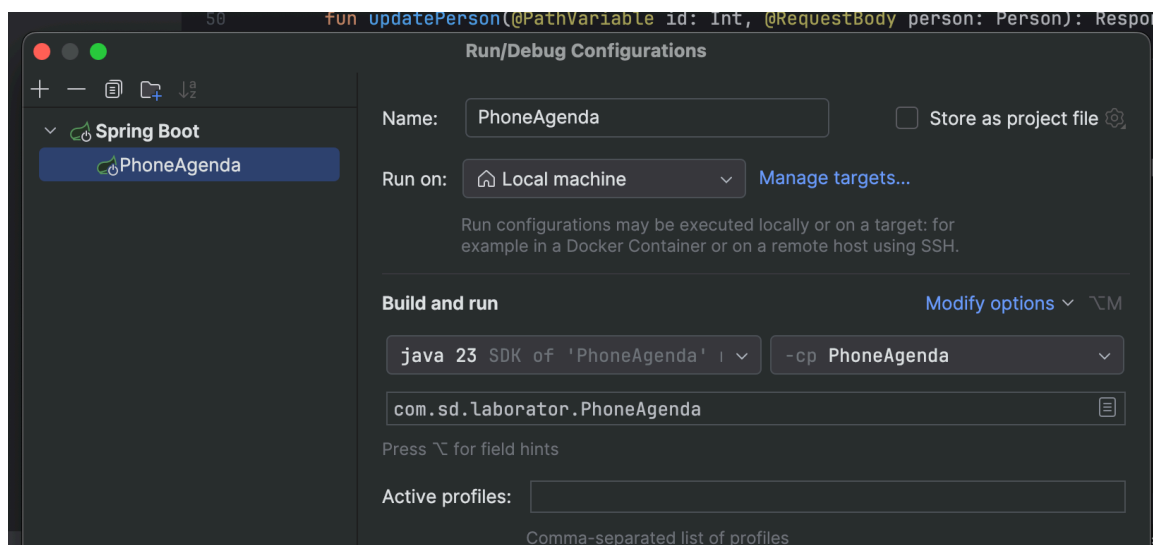
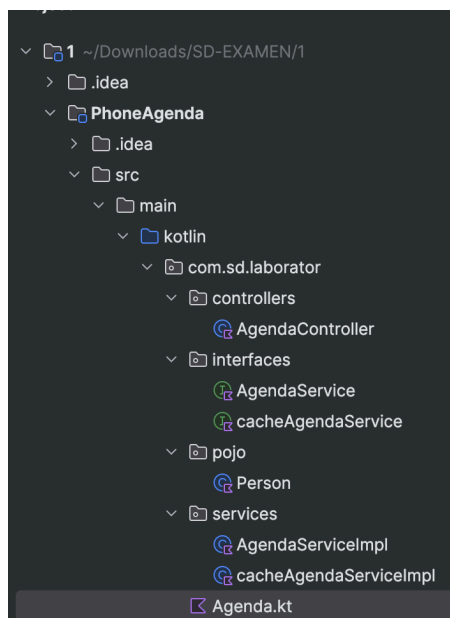


Pornind de la exemplul de implementare a agendei cu servicii REST din laborator să se realizeze următoarele:

- să se deseneze diagrama de clase;
- să se implementeze un serviciu (utilizand modelul portii API sau nu) | care asigură o metodă de cache ce răspunde dacă noua cerere apare în primele 30 de minute;

Daca se considera necesar se mai pot crea oricâte servicii se doreste conform gândirii studentului.

Se va utiliza orchestrarea si respectarea principiilor SOLID pentru microservicii. La prezentare se va arăta diagrama de clase si se va explica aceasta impreuna cu gradul de respectare a principillor. Apoi va urma discutia pe cod.



Notite

1. Dupa rulare accesezi localhost:8080/agenda
2. Pentru a vedea user-ul individual accesezi localhost:8080/person/1 sau 2 sau 3
3. Creezi o aplicatie springBoot(Vezi in laborator)

AgendaController

```
package com.sd.laborator.controllers
import com.sd.laborator.interfaces.*
import com.sd.laborator.pojo.Person
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.http.HttpStatus
import org.springframework.http.ResponseEntity
import org.springframework.web.bind.annotation.*
import java.time.LocalDateTime

@RestController
class AgendaController {
    @Autowired
    private lateinit var agendaService: AgendaService

    @Autowired
    private lateinit var cacheAgendaService: cacheAgendaService

    @RequestMapping(value = ["/person"], method = [RequestMethod.POST])
    fun createPerson(@RequestBody person: Person): ResponseEntity<Unit> {
        agendaService.createPerson(person)
        cacheAgendaService.createPerson(person)
        return ResponseEntity(Unit, HttpStatus.CREATED)
    }

    @RequestMapping(value = ["/person/{id}"], method = [RequestMethod.GET])
    fun getPerson(@PathVariable id: Int): ResponseEntity<Person?> {
        var person: Person? = cacheAgendaService.getPerson(id)

        var status = if (person == null) { HttpStatus.NOT_FOUND } else {
HttpStatus.OK }

        if(status == HttpStatus.OK) {
            println("Cache called")
            return ResponseEntity(person, status)
        }
        else {
            person = agendaService.getPerson(id)
            status = if (person == null) { HttpStatus.NOT_FOUND } else {
HttpStatus.OK }
```

```

        //daca s-a gasit in storage, il punem si in cache, pentru a-l putea
        accesa rapid
        if(status == HttpStatus.OK) {
            println("Not found in cache, but creating new record.")
            person!!.date_created = LocalDateTime.now() // il luam din
            storage si trebuie sa ii schimbam data.
            cacheAgendaService.createPerson(person)
        }
        return ResponseEntity(person, status)
    }
}

@RequestMapping(value = ["/person/{id}"], method = [RequestMethod.PUT])
fun updatePerson(@PathVariable id: Int, @RequestBody person: Person):
ResponseEntity<Unit> {
    agendaService.getPerson(id)?.let {
        agendaService.updatePerson(it.id, person)
        cacheAgendaService.updatePerson(it.id, person)
        return ResponseEntity(Unit, HttpStatus.ACCEPTED)
    } ?: return ResponseEntity(Unit, HttpStatus.NOT_FOUND)
}

@RequestMapping(value = ["/person/{id}"], method = [RequestMethod.DELETE])
fun deletePerson(@PathVariable id: Int): ResponseEntity<Unit> {
    if (agendaService.getPerson(id) != null) {
        agendaService.deletePerson(id)
        cacheAgendaService.deletePerson(id)
        return ResponseEntity(Unit, HttpStatus.OK)
    } else {
        return ResponseEntity(Unit, HttpStatus.NOT_FOUND)
    }
}

@RequestMapping(value = ["/agenda"], method = [RequestMethod.GET])
fun search(@RequestParam(required = false, name = "lastName", defaultValue =
"") lastName: String,
           @RequestParam(required = false, name = "firstName",
defaultValue = "") firstName: String,
           @RequestParam(required = false, name = "telephone",
defaultValue = "") telephoneNumber: String):
ResponseEntity<List<Person>> {
    val personList = agendaService.searchAgenda(lastName, firstName,
telephoneNumber)
    var httpStatus = HttpStatus.OK
    if (personList.isEmpty()) {
        httpStatus = HttpStatus.NO_CONTENT
    }
    return ResponseEntity(personList, httpStatus)
}

```

```
}  
}
```

AgendaService

```
package com.sd.laborator.interfaces  
  
import com.sd.laborator.pojo.Person  
  
interface AgendaService {  
    fun getPerson(id: Int) : Person?  
    fun createPerson(person: Person)  
    fun deletePerson(id: Int)  
    fun updatePerson(id: Int, person: Person)  
    fun searchAgenda(lastNameFilter: String, firstNameFilter: String,  
telephoneNumberFilter: String): List<Person>  
}
```

cacheAgendaService

```
package com.sd.laborator.interfaces  
  
import com.sd.laborator.pojo.Person  
  
interface cacheAgendaService {  
    fun getPerson(id: Int) : Person?  
    fun createPerson(person: Person)  
    fun deletePerson(id: Int)  
    fun updatePerson(id: Int, person: Person)  
}
```

Person

```
package com.sd.laborator.pojo  
  
import java.sql.Time  
import java.time.LocalDateTime  
  
data class Person(  
    var id: Int = 0,  
    var lastName: String = "",  
    var firstName: String = "",  
    var telephoneNumber: String = "",  
    var date_created: LocalDateTime = LocalDateTime.now()  
)
```

AgendaServiceImpl

```
package com.sd.laborator.services;

import com.sd.laborator.interfaces.AgendaService
import com.sd.laborator.pojo.Person
import org.springframework.stereotype.Service
import java.util.concurrent.ConcurrentHashMap

@Service
class AgendaServiceImpl : AgendaService {
    companion object {
        val initialAgenda = arrayOf(
            Person(1, "Hello", "Kotlin", "1234"),
            Person(2, "Hello", "Spring", "5678"),
            Person(3, "Hello", "Microservice", "9101112")
        )
    }

    private val agenda = ConcurrentHashMap<Int, Person>(
        initialAgenda.associateBy { person: Person -> person.id }
    )

    override fun getPerson(id: Int): Person? {
        return agenda[id]
    }

    override fun createPerson(person: Person) {
        agenda[person.id] = person
    }

    override fun deletePerson(id: Int) {
        agenda.remove(id)
    }

    override fun updatePerson(id: Int, person: Person) {
        deletePerson(id)
        createPerson(person)
    }

    override fun searchAgenda(lastNameFilter: String, firstNameFilter: String,
        telephoneNumberFilter: String): List<Person> {
        return agenda.filter {
            it.value.lastName.toLowerCase().contains(lastNameFilter, ignoreCase
= true) &&
                it.value.firstName.toLowerCase().contains(firstNameFilter,
ignoreCase = true) &&
                it.value.telephoneNumber.contains(telephoneNumberFilter)
        }.map {
```

```

        it.value
    }.toList()
}
}

```

cacheAgendaService

```

package com.sd.laborator.services;
import com.sd.laborator.interfaces.cacheAgendaService
import com.sd.laborator.pojo.Person
import org.springframework.stereotype.Service
import java.time.LocalDateTime
import java.time.temporal.ChronoUnit
import java.util.concurrent.ConcurrentHashMap

@Service
class cacheAgendaServiceImpl : cacheAgendaService {
    companion object {
        val initialAgenda = arrayOf(
            Person(1, "Hello", "Kotlin", "1234"),
            Person(2, "Hello", "Spring", "5678"),
            Person(3, "Hello", "Microservice", "9101112")
        )
    }

    private val agenda = ConcurrentHashMap<Int, Person>(
        initialAgenda.associateBy { person: Person -> person.id }
    )

    override fun getPerson(id: Int): Person? {
        for(contact in agenda) // verificare cache si stergere daca
        {
            if(contact.value.date_created.until(LocalDateTime.now(),
ChronoUnit.MINUTES) >= 1)
                deletePerson(contact.value.id)
        }

        agenda[id]?.date_created = LocalDateTime.now()
        return agenda[id]
    }

    override fun createPerson(person: Person) {
        for(contact in agenda) // verificare cache si stergere daca
        {
            if(contact.value.date_created.until(LocalDateTime.now(),
ChronoUnit.MINUTES) >= 1)
                deletePerson(contact.value.id)
        }
    }
}

```

```

        agenda[person.id]?.date_created = LocalDateTime.now()
        agenda[person.id] = person
    }

    override fun deletePerson(id: Int) {
        agenda.remove(id)
    }

    override fun updatePerson(id: Int, person: Person) {
        for(contact in agenda) // verificare cache si stergere daca
        {
            if(contact.value.date_created.until(LocalDateTime.now(),
ChronoUnit.MINUTES) >= 1)
                deletePerson(contact.value.id)
        }

        deletePerson(id)
        createPerson(person)
    }
}

```

Agenda

```

package com.sd.laborator

import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
open class PhoneAgenda

fun main(args: Array<String>) {
    runApplication<PhoneAgenda>(*args)
}

```

Pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

    <modelVersion>4.0.0</modelVersion>

```

```
<groupId>com.sd.laborator</groupId>
<artifactId>PhoneAgenda</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>com.sd.laborator PhoneAgenda</name>

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.1.9.RELEASE</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <kotlin.version>1.3.61</kotlin.version>
  <kotlin.code.style>official</kotlin.code.style>
  <junit.version>4.12</junit.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.jetbrains.kotlin</groupId>
    <artifactId>kotlin-stdlib</artifactId>
    <version>${kotlin.version}</version>
  </dependency>
  <dependency>
    <groupId>org.jetbrains.kotlin</groupId>
    <artifactId>kotlin-test-junit</artifactId>
    <version>${kotlin.version}</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.jetbrains.kotlin</groupId>
    <artifactId>kotlin-reflect</artifactId>
    <version>${kotlin.version}</version>
  </dependency>
  <dependency>
```



```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <version>${project.parent.version}</version>
    </dependency>
</dependencies>

<build>
    <sourceDirectory>src/main/kotlin</sourceDirectory>
    <testSourceDirectory>src/test/kotlin</testSourceDirectory>

    <plugins>
        <plugin>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-plugin</artifactId>
            <version>${kotlin.version}</version>
            <executions>
                <execution>
                    <id>compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <compilerPlugins>
                            <plugin>spring</plugin>
                        </compilerPlugins>
                    </configuration>
                </execution>
                <execution>
                    <id>test-compile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>test-compile</goal>
                    </goals>
                </execution>
            </executions>
            <dependencies>
                <dependency>
                    <groupId>org.jetbrains.kotlin</groupId>
                    <artifactId>kotlin-maven-allopen</artifactId>
                    <version>${kotlin.version}</version>
                </dependency>
            </dependencies>
        </plugin>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>

```

```
</build>
```

```
</project>
```